FINAL SEMESTER PROJECT

Expense Tracker for A.Z. Fashion

Submitted By:
Eman Ahmad (FA25-BBD-037)

May 21, 2026

Contents

Expense Tracker Management System

Programming Fundamentals Semester Project Report

1. Introduction

Managing business expenses is one of the most important tasks for small businesses. Many small entrepreneurs still use notebooks or manual records to track their daily expenses and profits. This method often creates confusion, calculation mistakes, and difficulty in financial planning.

This project, named **Expense Tracker Management System**, was developed to solve the problem of manual expense management for small businesses. The system helps users record, organize, and manage business expenses digitally using Python programming concepts.

The project was designed while studying the workflow of a home-based boutique business named **A.Z. Fashion**, which manages customized formal dresses. The business owner faced difficulties in tracking fabric costs, accessory expenses, tailoring charges, and other daily operational expenses.

Without a digital system, it became difficult to:

- Maintain proper financial records
- Calculate profit and loss accurately
- Track monthly expenses
- Separate business expenses from personal expenses
- Analyze spending patterns

The Expense Tracker Management System provides a simple and organized solution for recording and managing expenses efficiently.

Real-World Relevance

Small businesses and home-based entrepreneurs usually do not have advanced accounting software because such systems can be expensive and difficult to use. This project provides a lightweight and affordable solution using basic programming concepts.

The system can be useful for:

- Small boutiques
- Freelancers
- Online sellers

- Home-based businesses
- Students managing budgets
- Shop owners

Objectives of the System

The main objectives of the system are:

- To create a digital expense tracking application
- To store expense records permanently using files
- To calculate and display expense summaries
- To organize expenses into categories
- To improve financial management for small businesses
- To apply programming concepts learned during the semester
- To build a real-world Python application

2. Problem Statement

Existing Problem

The selected business, A.Z. Fashion, was managing all expenses manually in notebooks. The owner recorded expenses by hand, which created several issues.

Difficulties Faced Without the System

The following problems were identified:

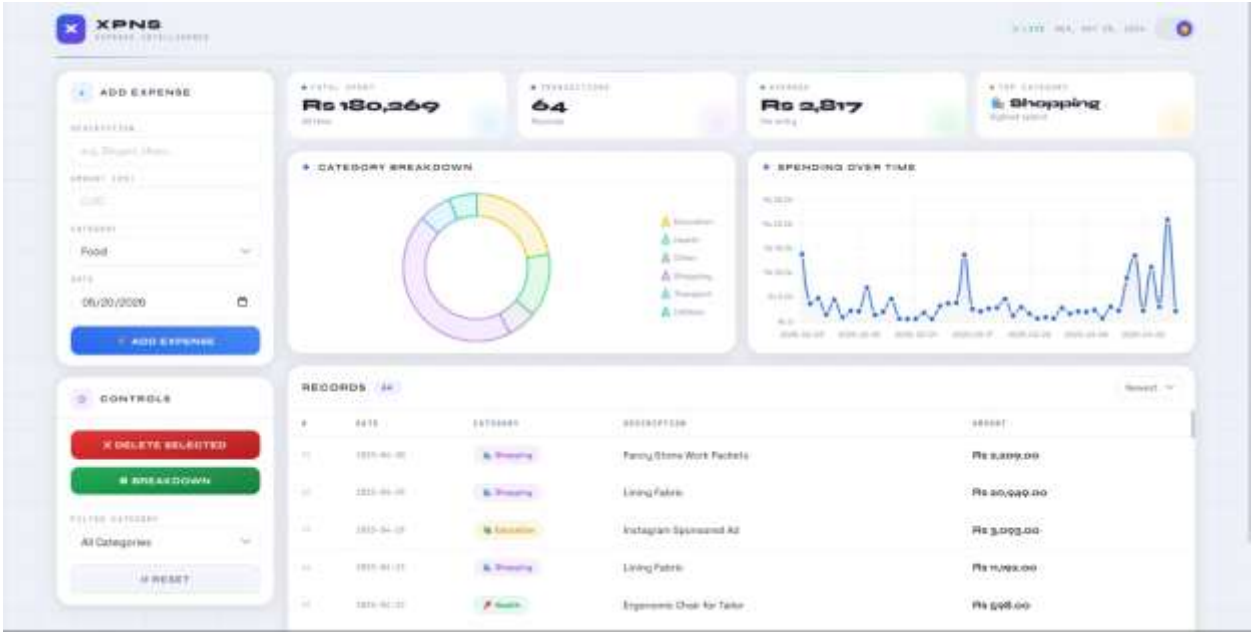
- Small expenses were often forgotten
- Manual calculations caused mistakes
- Monthly expense summaries were difficult to prepare
- Profit and loss calculations were inaccurate
- Business and household expenses were mixed together
- Searching old records was time-consuming
- Data could easily be lost or damaged

Proposed Solution

The Expense Tracker Management System was developed to solve these problems by:

- Digitally storing all expense records
- Organizing expenses into categories
- Providing summaries and reports
- Allowing users to add, update, delete, and view expenses
- Reducing human errors in calculations
- Improving financial planning and management

The system makes expense management faster, more organized, and more reliable.



3. Objectives of the System

The project was developed with the following objectives:

1. To design a user-friendly expense tracking system.
2. To maintain proper financial records digitally.
3. To reduce manual calculation errors.
4. To provide categorized expense management.
5. To implement file handling for permanent data storage.

6. To use object-oriented programming concepts.
7. To improve programming and problem-solving skills.
8. To build a practical application with real-world use.

4. System Features

The Expense Tracker Management System includes several important features.

4.1 Add Expense

The user can add new expenses by entering:

- Expense title
- Category
- Amount
- Date
- Description

The data is saved permanently in files.

```
  #  FUNCTION: Add a new expense
    # ──────────────────────────────────────────────
    def add_expense(self, description: str, amount, category: str, date: str)
-> Expense:
        """
        Validates inputs, creates Expense, appends to list, saves.
        EXCEPTION HANDLING: Raises ValueError on bad input.
        Returns the created Expense object.
        """
        if not description or not description.strip():
            raise ValueError("Description cannot be empty.")

        try:
            amount = float(amount)
        except (TypeError, ValueError):
            raise ValueError(f"Invalid amount: '{amount}'. Must be a number.")

        if amount <= 0:
            raise ValueError("Amount must be greater than zero.")

        if category not in CATEGORIES:
            raise ValueError(f"Invalid category: '{category}'.")

        try:
            datetime.strptime(date, "%Y-%m-%d")
        except ValueError:
            raise ValueError("Date must be in YYYY-MM-DD format.")
```

```
        expense = Expense(description, amount, category, date)
        self.expenses.append(expense)     # CONCEPT: List — append
        self._save_to_file()
        return expense
```

4.2 View Expenses

Users can view all saved expense records in an organized format.

```
    #  FUNCTION: Get all expenses (as dicts)
    # ──────────────────────────────────────────────
    def get_all(self) -> list:
        """Returns all expenses as a list of dictionaries."""
        # CONCEPT: List — iterate and convert each object
        return [e.to_dict() for e in self.expenses]
```

4.3 Search Expenses

The system allows users to search expenses based on:

- Category
- Date
- Expense Title

```
#  FUNCTION: Filter by category
    # ──────────────────────────────────────────────
    def filter_by_category(self, category: str) -> list:
        """
        CONCEPT: List — returns filtered subset by category.
        Returns all if category is 'All'.
        """
        if category == "All":
            return self.get_all()
        return [e.to_dict() for e in self.expenses if e.category == category]
```

- xpense title

4.4 Delete Expense

Users can remove incorrect or unnecessary records from the system.

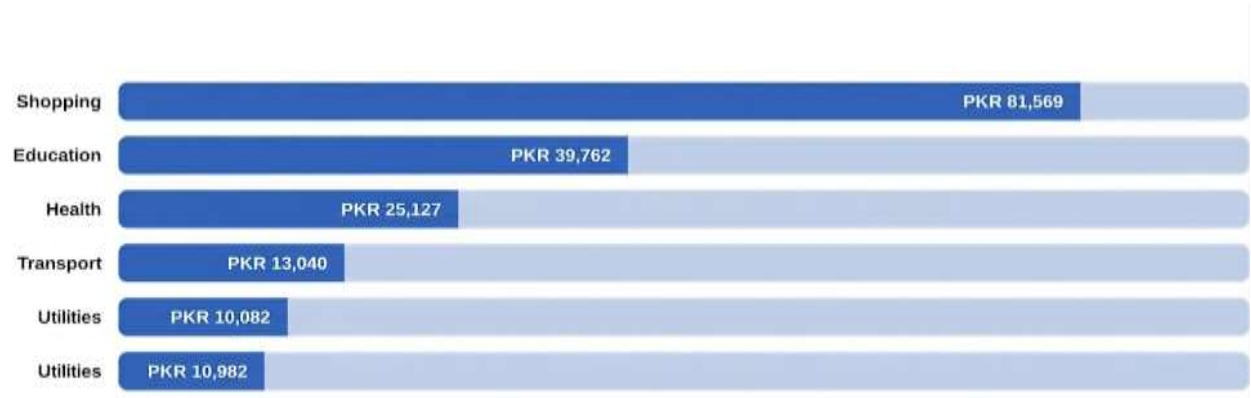
```
    def delete_expense(self, expense_id: int) -> bool:
        """
        Removes the expense with the given ID from the list.
        EXCEPTION HANDLING: Returns False if ID not found.
        """
        original_count = len(self.expenses)
        # CONCEPT: List comprehension — filter out the deleted item
        self.expenses = [e for e in self.expenses if e.expense_id != expense_id]

        if len(self.expenses) == original_count:
            return False   # Nothing was removed
```

```
        self._save_to_file()
        return True
```

4.5 Expense Summary

The system calculates:

- Total expenses
- Category-wise expenses
- Monthly expense summaries



4.6 File Storage

All data is stored using file handling so that records remain available even after restarting the program.

4.7 User-Friendly Menu

A menu-driven interface helps users navigate the program easily.

4.8 Error Handling

The system handles invalid input properly using exception handling techniques.

5. Concepts Used in the Project

This project uses multiple programming concepts learned during the semester.

Technologies and Concepts Used

Concept	Where Used
Dictionaries	Category summaries and JSON data handling
Lists	Storing expense records
Functions	Flask route handlers and helper functions
Exception Handling	File handling and input validation
File Handling	Reading and writing expenses.json
Object-Oriented Programming (OOP)	Expense and ExpenseManager classes
Flask Framework	Web application routing and HTML templates

5.1 Functions

Functions were used to divide the program into smaller logical sections. This improved code readability and reduced repetition.

Examples:

- add_expense()
- view_expenses()
- search_expense()
- delete_expense()
- calculate_summary()

Benefits of Using Functions

- Easier debugging
- Better code organization
- Improved readability
- Reusable code

5.2 Lists

Lists were used to store multiple expense records temporarily during program execution.

Example:

expenses = []

Lists helped manage collections of expense data efficiently.

5.3 Dictionaries

Dictionaries were used to store individual expense details in key-value pairs.

Example:


```
{
    "id": 1778408695807,
    "description": "Instagram Sponsored Ad",
    "amount": 5830.0,
    "category": "Education",
    "date": "2025-02-03"
}
```


Advantages

- Organized data storage
- Easy access to values
- Better data management

5.4 Object-Oriented Programming (OOP)

Object-oriented programming concepts were used to improve the structure of the system.

Classes and objects helped represent real-world entities such as expenses and trackers.

Example:

```
class Expense:
    """Represents a single expense entry."""
```

```
    def __init__(self, description: str, amount: float, category: str,
                 date: str = None, expense_id: int = None):
        self.expense_id  = expense_id or int(datetime.now().timestamp() * 1000)
        self.description = description.strip()
        self.amount      = float(amount)
        self.category    = category
        # Default to today's date if none provided
        self.date = date or datetime.today().strftime("%Y-%m-%d")
```

OOP Benefits

- Better code structure
- Improved maintainability
- Easier scalability
- Real-world modeling

5.5 File Handling

File handling was used to save and retrieve expense records permanently.

The program uses files to:

- Save expense data
- Load existing records
- Maintain data after restarting the application

5.6 Exception Handling

Exception handling was used to prevent the program from crashing due to invalid input.

Example:

```
try:
    amount = float(input("Enter amount: "))
except ValueError:
    print("Invalid input")
```

Benefits

- Prevents crashes
- Improves user experience
- Handles invalid input safely

6. System Design and Workflow

6.1 System Workflow

The system follows the following workflow:

1. User starts the application
2. Main menu is displayed
3. User selects an option
4. Program performs the required task
5. Data is saved or updated
6. Results are displayed
7. User returns to the main menu or exits

6.2 Menu Structure

1. Add Expense

2. View Expenses

3. Search Expense

4. Delete Expense

5. View Summary

6. Exit

6.3 Flowchart

```
Start
   |
Display Menu
   |
User Selects Option
   |
Perform Required Function
   |
Save/Update Data
   |
Display Result
   |
Return to Menu
   |
Exit
```

6.4 Module Interaction

The project contains different modules that interact together:

Module	Purpose
Input Module	Takes user input
Expense Module	Manages expense records
File Module	Stores and retrieves data
Summary Module	Calculates expense reports
Validation Module	Handles invalid inputs

---

7. Code Explanation

Only important sections of the code are explained below.

7.1 Adding Expense

This section allows users to enter new expense information.

```
expense = {
"title": title,
"category": category,
"amount": amount
}
expenses.append(expense)
```

Explanation

- A dictionary stores expense details.
- The expense is added to the expense list.
- Data is later saved to a file.

7.2 File Saving Logic

```
with open("expenses.txt", "a") as file:
file.write(str(expense) + "\n")
```

Explanation

- Opens the file in append mode.
- Saves new expense records.

- Prevents old data from being overwritten.

7.3 Expense Summary Calculation

```
total = 0
for expense in expenses:
total += expense["amount"]
```

Explanation

- Loops through all expenses.
- Adds all amounts together.
- Displays the total expense.

7.4 Exception Handling

```
try:
amount = float(input("Enter amount: "))
except ValueError:
print("Please enter a valid number")
```

Explanation

- Handles invalid user input.
- Prevents the application from crashing.

8. File Handling and Data Storage

The project uses file handling to store expense data permanently.

File Operations Used

Operation	Purpose
Read Mode	Load saved records
Write Mode	Create new files
Append Mode	Add new records

Advantages of File Storage

- Permanent data storage
- Easy retrieval of records
- Better data management

- No hardcoding of data

9. Error Handling and Validation

The project includes validation techniques to improve reliability.

Validation Features

- Prevents empty input
- Prevents invalid numeric values
- Handles incorrect menu options
- Displays user-friendly error messages

Importance

These techniques ensure the system works smoothly without unexpected crashes.

10. Testing and Results

The system was tested using multiple expense records.

Test Cases Performed

Test Case	Result
Add expense	Successful
Save data to file	Successful
View expenses	Successful
Delete expense	Successful
Invalid input handling	Successful
Summary calculation	Successful

System Output

The system successfully:

- Stored expense records
- Displayed organized data

- Calculated totals correctly
- Prevented crashes during invalid input
- Maintained data using files

11. Conclusion

The Expense Tracker Management System successfully solved the problem of manual expense management for small businesses.

The project provided an organized and digital method for storing and managing expenses. It reduced calculation errors, improved financial tracking, and simplified record management.

What Was Achieved

- A fully functional Python application was developed.
- Expense records were managed digitally.
- File handling was implemented successfully.
- Real-world business problems were addressed.

Skills Learned

During this project, the following skills were improved:

- Python programming
- Problem-solving
- Object-oriented programming
- File handling
- Exception handling
- Program organization
- Real-world software development

This project also improved understanding of how programming can be used to solve practical business problems.

12. References

Python Official Documentation
https://docs.python.org/3/

W3Schools Python Tutorials
https://www.w3schools.com/python/

GeeksforGeeks Python Programming
https://www.geeksforgeeks.org/python-programming-language/

Course Notes and Class Lectures

Jupyter Notebook Documentation
https://jupyter.org/